

MODULE 8 · PACKAGING & EXTENSIBILITY

Custom Resource Definitions

Extending the API with new kinds — and the controllers that act on them

Kubernetes' API isn't fixed — you can extend it

Built-in Kinds like Pod and Deployment cover generic workloads, but real systems need domain-specific objects — a certificate, a database, a message queue.

A **CustomResourceDefinition (CRD)** registers a brand-new Kind without forking Kubernetes. Whole ecosystems — cert-manager, Argo, Prometheus Operator — are built this way.

BY THE END YOU CAN

- Explain CRD (schema) vs CR (instance)
- Read a CRD's group, Kind and scope
- Explain the controller/operator pattern
- Create a cert-manager Certificate and watch it reconcile

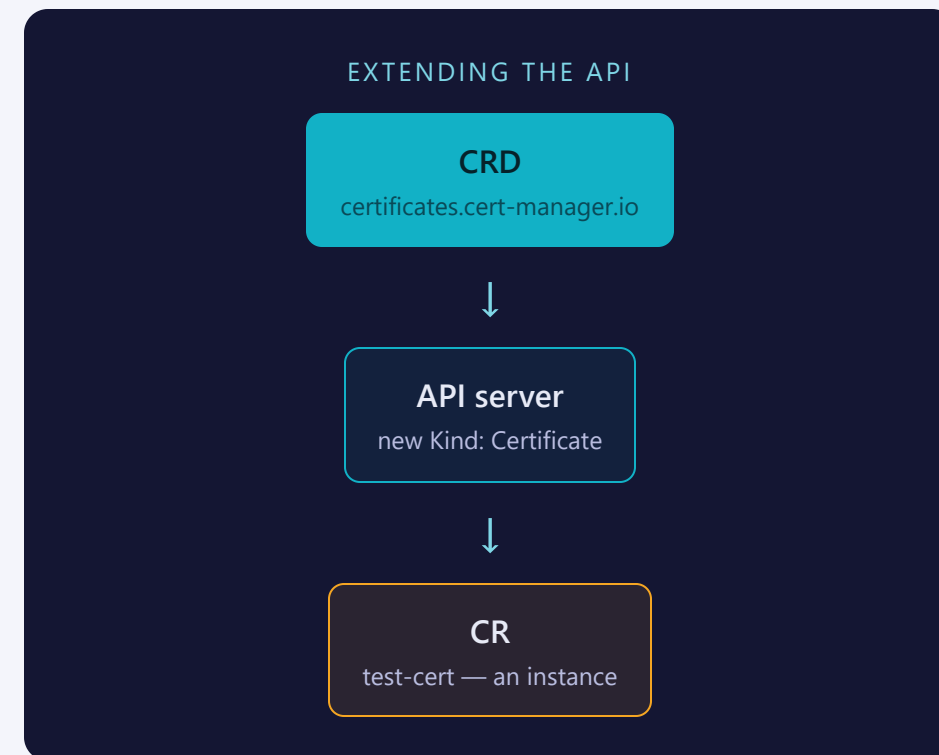
CONCEPT

A CRD registers the Kind; nothing acts on it yet

A CRD tells the API server about a new **Kind** — its group, version, field schema and scope — the same way Pod was already registered, just installed later instead of built in.

Once it exists, that Kind behaves like any other: `kubectl get`, `apply`, RBAC and validation all just work.

The CRD object itself is cluster-scoped; the Kind it defines can be namespaced or cluster-scoped.



One schema, many instances

	CRD — THE SCHEMA	CR — THE INSTANCE
What it is	A new Kind's definition: fields, validation, scope	An actual object of that Kind
Who creates it	A platform or tool author (cert-manager's own manifests)	You, or an app developer
cert-manager example	<code>certificates.cert-manager.io</code>	a <code>Certificate</code> named <code>test-cert</code>
Analogy	A class definition	An object created from that class

MENTAL MODEL

A CRD adds a new noun to the cluster's vocabulary; a CR is one sentence written using that noun.

A CR looks like any other manifest

```
apiVersion: cert-manager.io/v1 # the CRD's group/version
kind: Certificate               # the Kind the CRD registered
metadata:
  name: test-cert
  namespace: training
spec:
  secretName: test-cert-tls # where the issued cert lands
  issuerRef:
    name: selfsigned-issuer
    kind: ClusterIssuer # another CRD's Kind
  dnsNames:
    - test.example.com
  duration: 2160h # cert validity window
  renewBefore: 360h # renew 15 days before expiry
```

```
$ kubectl get crd | grep cert-manager
$ kubectl get certificate test-cert -n training
```

- **apiVersion / kind** — same shape, just a custom group
- **metadata.namespace** — Certificate is namespaced, like a Pod
- **issuerRef** — points at a ClusterIssuer, itself a CR
- **duration / renewBefore** — cert-manager schedules renewal for you
- **status** (not shown) — where the controller writes results back

No imperative shortcut for custom Kinds

BUILT-IN KIND (POD)

```
kubectl run web --image=nginx:1.25
```

Imperative helper builds the spec for you

CUSTOM RESOURCE (CERTIFICATE)

no generator — `kubectl create certificate` doesn't exist

You always write or copy the YAML yourself

THE BRIDGE TRICK

`kubectl`'s imperative generators are hardcoded to built-in Kinds. For a CRD, `apply` is the only path — but you can still validate before committing:

```
$ kubectl apply -f certificate.yaml --dry-run=server
# checked against the CRD's own schema, nothing created
```

SEE IT

Watching the controller do its job

`kubectl apply` only queues the request — it doesn't mean the certificate exists yet. `kubectl wait` blocks until cert-manager's controller has actually reconciled it.

READY: True means the controller has issued the cert and written the Secret — not just that the object was accepted.

```
$ kubectl wait --for=condition=Ready \
    certificate/test-cert -n training --timeout=60s
certificate.cert-manager.io/test-cert condition met
```

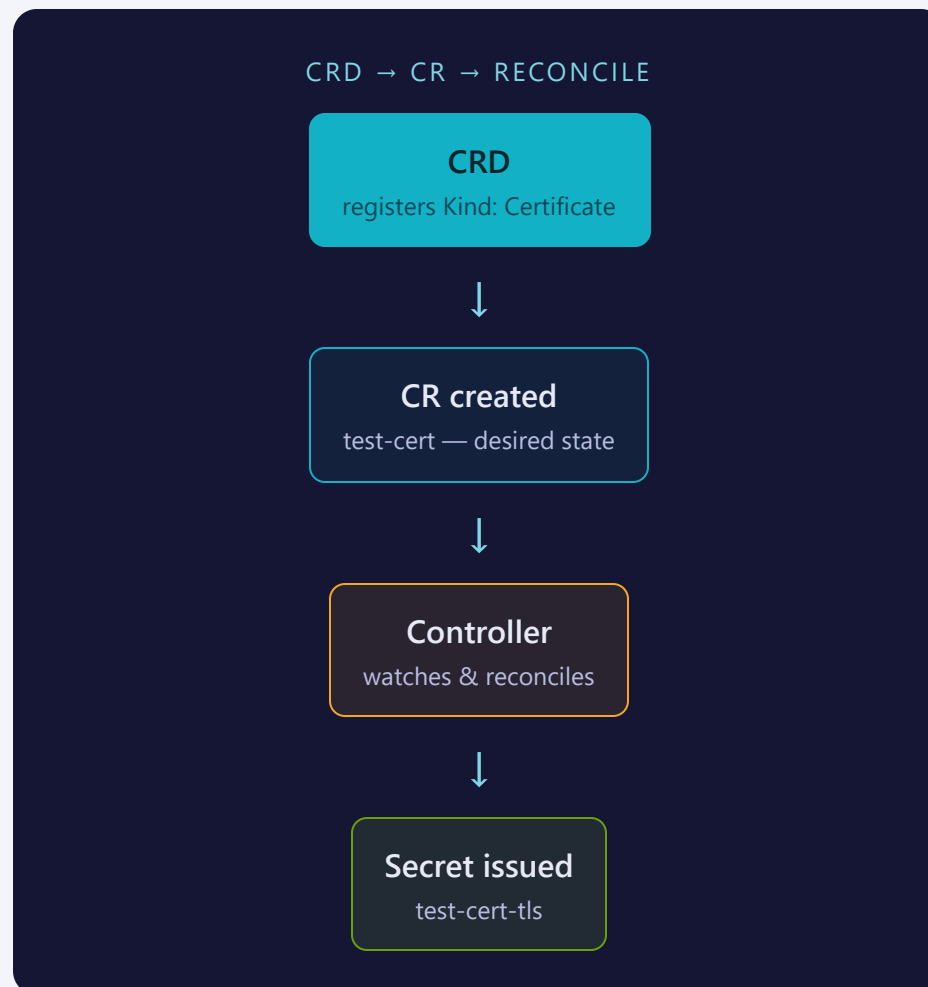
```
$ kubectl get certificate test-cert -n training
```

NAME	READY	SECRET	AGE
test-cert	True	test-cert-tls	12s

A controller closes the loop between spec and reality

Installing a CRD only adds a shape to the API — nothing acts on it by itself. A **controller** (the cert-manager operator, here) watches for objects of that Kind and drives real state to match.

This watch-and-reconcile loop is what "operator" means: operational knowledge a human would otherwise apply by hand, encoded as code.



Where people trip

- **CRD not installed yet.** Applying a Certificate before cert-manager's CRDs exist fails with `no matches for kind "Certificate"`.
- **No controller running.** A CR with no matching controller Pod just sits there — the Kind exists, but nothing reconciles it.
- **Trusting get alone.** A blank or `False` `READY` hides the reason — `describe` shows the controller's events.
- **Deleting the CRD.** Removing it deletes every CR of that Kind, cluster-wide — a far bigger blast radius than one object.

Commands to keep at hand

```
$ kubectl get crd                # every custom Kind installed
$ kubectl explain certificate.spec # schema, straight from the CRD
$ kubectl get certificates -A     # CRs across namespaces
$ kubectl describe certificate NAME -n NS # the controller's events
$ kubectl get secret NAME -n NS -o yaml # what the operator produced
```

Tip: `kubectl explain` reads live off the installed CRD — always current, no docs to go stale.

RECAP

CRDs in one breath

- CRD registers a new Kind; a CR is one instance of it
- No CRD installed → the Kind doesn't exist → `apply` fails
- A controller/operator watches CRs and reconciles real state
- `cert-manager`'s Certificate → Secret is the operator pattern in action

HANDS-ON

Now run Lab 8.3
`labs/8.3/`

Next: [9.1 — Pod Failures](#)